

Chapter 2.

Natural Language Processing

- | 1.1. What is NLP?
- | 1.2. Introduction to Search
- | 1.3. Processing text
- | 1.4. Representing text
- | 1.5. Similarity measures
- | 1.6. Next up

- Understand what NLP is and its real-world applications
- Explain why text is challenging for computers
- Perform text preprocessing (cleaning, tokenization, normalization)
- Implement keyword search (simple and TF-based)
- Start to understand sparse vs dense vector
- Implement similarity-based search using TF-IDF
- Cluster documents using machine learning (in order to visualize similarity)

NLP = Teaching computers to **understand, interpret, and generate human language**

 **Search engines:** Understanding what you really mean when you search, and finding it

 **Email filtering:** Identifying spam vs important messages

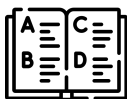
 **Chatbots:** Understanding and replying to customer questions

 **Document analysis:** Organizing and finding information in large document collections

 **Recommendation systems:** Finding similar content (movies, products, jobs)

... so much more

NLP Use cases



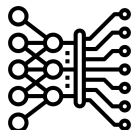
Representative
models



Retrievals



Text Classification



Generative
models



Text translation



Text generation

Let's start with a demo

Let's start with a demo

You're building a system to help recruiters search through thousands of CVs/resumes.

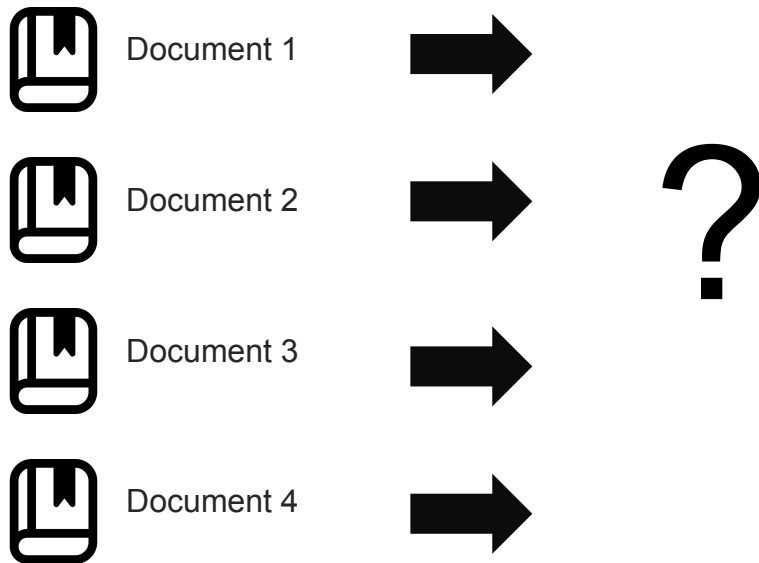
The Challenge: A recruiter searches for "**Python developer with machine learning experience**" but needs to find candidates who wrote:

- "Proficient in Python and ML"
- "Experienced in data science using Python"
- "Strong background in artificial intelligence and Python programming"

Simple keyword matching fails! The same meaning is expressed differently.

NLP = Teaching computers to **understand, interpret, and generate** human language





I want to look for movies about space adventures

Will I enjoy this movie?



Movie 1



Movie 2



Movie 3



| Syntactic models

Syntactic models focus on the structure and form of language, analyzing how words and symbols are arranged according to grammatical or formal rules.

| Semantic models

Semantic models capture meaning and relationships between concepts, interpreting the content and context of language beyond its surface structure.

| Syntactic models

"Why Is The Rum Always Gone?"



Why	is	the	rum	always	gone	Other words
1	1	1	1	1	1	...

| Syntactic models



Document 1



w1	w2	w3	w4	w5	...	wn
1	0	1	0	1	...	1



Document 2



w1	w2	w3	w4	w5	...	wn
0	3	1	0	1	...	1



Document 3



w1	w2	w3	w4	w5	...	wn
1	0	1	0	1	...	1



Document 4



w1	w2	w3	w4	w5	...	wn
1	0	1	0	1	...	1

Bag of Words

The **Bag-of-Words (BoW) Model** is a feature extraction technique in NLP that converts text into **numerical vectors** by simply **counting the occurrence of each word** in a document. This vector representation allows machine learning algorithms to process text data. Its main limitation is that it **completely ignores word order and grammatical structure**.



I Syntactic models

Simple but they “understand” only syntax.

| Syntactic models

Movie 1: An amazing creature **is running** from a hunter.

['an': 1, 'amazing': 1, 'creature': 1, 'is': 1, 'running': 1, 'from': 1, 'a': 1, 'hunter': 1, ' ': 1]

Movie 2: The hero **ran** through the city to **escape**.

['the': 2, 'hero': 1, 'ran': 1, 'through': 1, 'city': 1, 'to': 1, 'escape': 1, ' ': 1]

Movie 3: **Running** away, the villain **escapes** the **runners!**

['running': 1, 'away': 1, ' ': 1, 'the': 2, 'villain': 1, 'escapes': 1, 'runners!': 1, ' ': 1]

Movie 4: A fun, but slightly **boring**, fantasy film.

['a': 1, 'fun': 1, ' ': 1, 'but': 1, 'slightly': 1, 'boring': 1, 'fantasy': 1, 'film': 1, ' ': 1]

Chapter 2.

Natural Language Processing

- | 1.1. What is NLP?
- | 1.2. Introduction to Search
- | 1.3. Processing text
- | 1.4. Representing text
- | 1.5. Similarity measures
- | 1.6. Next up



Formally speaking we could define search as:

$$F(q,c) = y$$

I.e a function that given a **query** q and a **corpus** c we get a set of **relevant** results y

Then, building a good search system means **optimizing** each part:

- crafting the right **query representation** q ,
- structuring or *embedding* the **corpus** c effectively,
- and designing or learning the best **retrieval function** F

Query: How the user expresses intent, can we make it **clearer**, **richer**, or **smarter**?

Corpus: How the information is **stored** and **represented**, can we make it easier to match, faster to search, or richer in features?

Function: How we **match** and **rank**, can we use better algorithms, learning signals, or hybrid strategies?

Query (q)	Corpus (c)	Retrieval Function (F)
Text representation / embeddings	Raw vs. processed content	Matching strategy (keywords, vectors)
Query expansion / synonyms	Indexing (inverted, vector)	Ranking (TF-IDF, BM25, learned)
Contextualization (history, intent)	Preprocessing / cleaning	Learning & adaptation (ML, feedback loops)
Structured queries / filters	Metadata / structure	Hybrid approaches (lexical + semantic)

Chapter 2.

Natural Language Processing

- | 1.1. What is NLP?
- | 1.2. Introduction to Search
- | 1.3. Processing text
- | 1.4. Representing text
- | 1.5. Similarity measures
- | 1.6. Next up

| Syntactic models

Movie 1: An amazing creature **is running** from a hunter.

['an': 1, 'amazing': 1, 'creature': 1, 'is': 1, 'running': 1, 'from': 1, 'a': 1, 'hunter': 1, ' ': 1]

Movie 2: The hero **ran** through the city to **escape**.

['the': 2, 'hero': 1, 'ran': 1, 'through': 1, 'city': 1, 'to': 1, 'escape': 1, ' ': 1]

Movie 3: **Running** away, the villain **escapes** the **runners!**

['running': 1, 'away': 1, ' ': 1, 'the': 2, 'villain': 1, 'escapes': 1, 'runners!': 1, ' ': 1]

Movie 4: A fun, but slightly **boring**, fantasy film.

['a': 1, 'fun': 1, ' ': 1, 'but': 1, 'slightly': 1, 'boring': 1, 'fantasy': 1, 'film': 1, ' ': 1]

| Syntactic models

Vocabulary

all unique words in a corpus; each word becomes a feature in syntactic models

['an', 'amazing', 'creature', 'is', 'running', 'from', 'a', 'hunter', '.', 'the', 'hero', 'ran', 'through', 'city', 'to', 'escape', 'away', ',', 'villain', 'escapes', 'runners!', '!', 'fun', 'but', 'slightly', 'boring', 'fantasy', 'film']

| Syntactic models

Vocabulary

all unique ~~words~~ tokens in a *corpus*; each word becomes a feature in syntactic models

['an', 'amazing', 'creature', 'is', 'running', 'from', 'a', 'hunter', '.', 'the', 'hero', 'ran', 'through', 'city', 'to', 'escape', 'away', ',', 'villain', 'escapes', 'runners!', '!', 'fun', 'but', 'slightly', 'boring', 'fantasy', 'film']

| Syntactic models

Vocabulary

all unique words tokens in a corpus; each token becomes a feature in syntactic models

['an', 'amazing', 'creature', 'is', 'running', 'from', 'a', 'hunter', '.', 'the', 'hero', 'ran', 'through', 'city', 'to', 'escape', 'away', ',', 'villain', 'escapes', 'runners!', '!', 'fun', 'but', 'slightly', 'boring', 'fantasy', 'film']

An amazing creature is running from a hunter

[1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

| Syntactic models

But, are all those **tokens** necessary?

all unique ~~words~~ tokens in a corpus; each word becomes a feature in syntactic models

['an', 'amazing', 'creature', 'is', 'running', 'from', 'a', 'hunter', '.', 'the', 'hero', 'ran', 'through', 'city', 'to', 'escape', 'away', ',', 'villain', 'escapes', 'runners!', '!', 'fun', 'but', 'slightly', 'boring', 'fantasy', 'film']

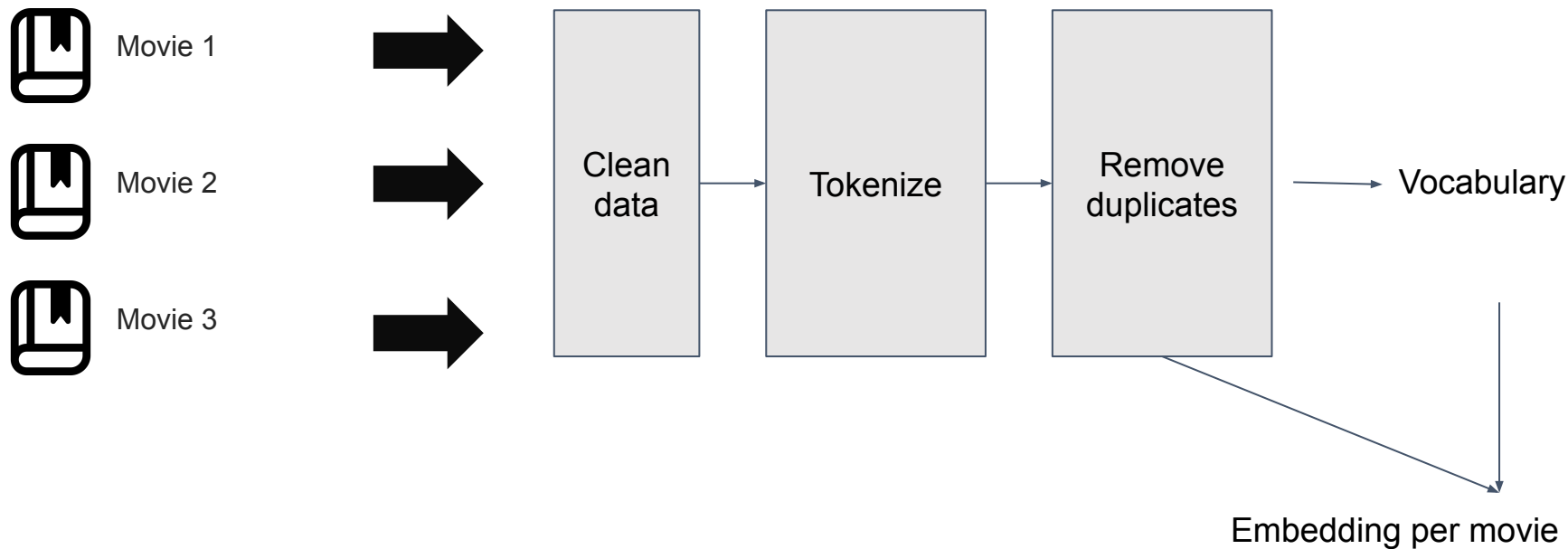
| Syntactic models

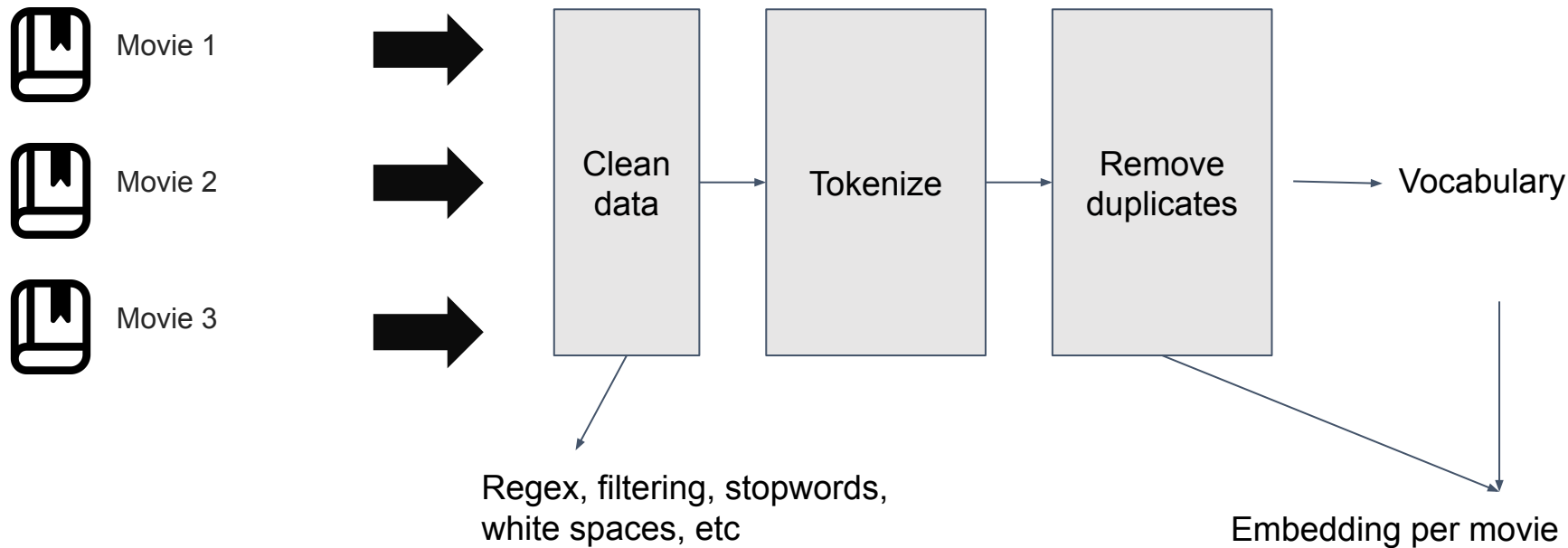
But, are all those **tokens** necessary?

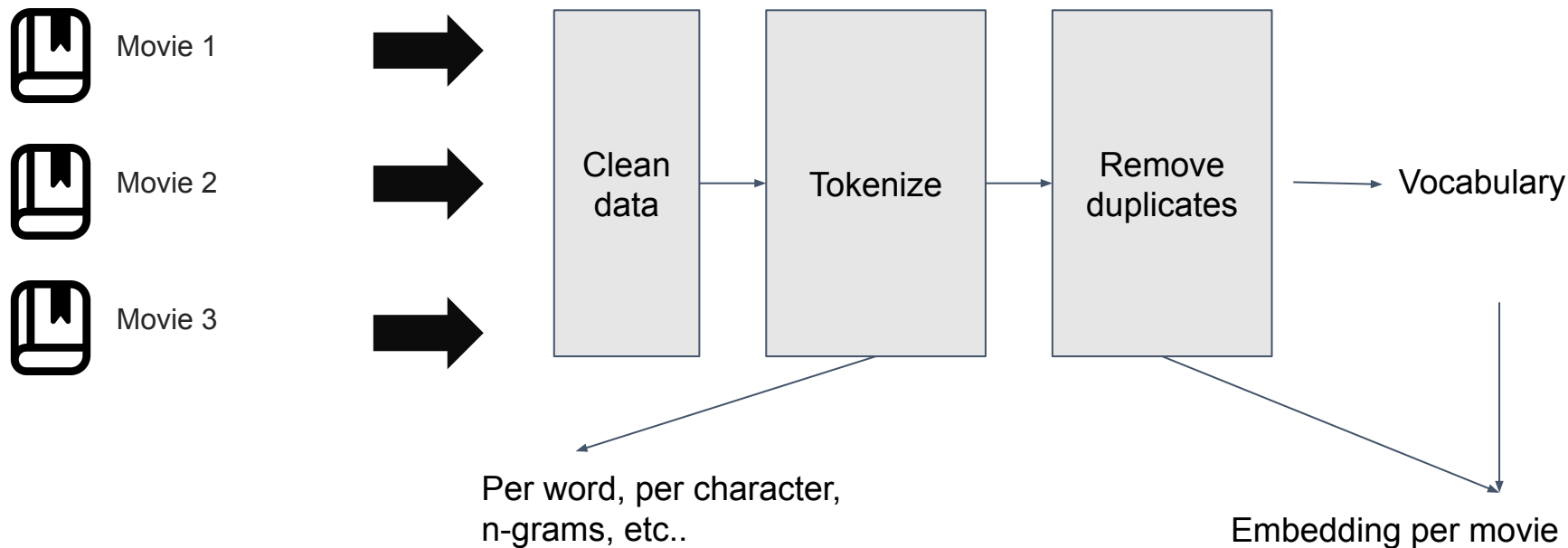
Not really. There are two standard ways to handle this in syntactic models:

Stemming is the process of reducing a word to its base or root form by removing prefixes or suffixes — often using simple, rule-based heuristics (e.g., *"running"* → *"run"*, *"flies"* → *"fli"*).

Lemmatization is the process of reducing a word to its canonical or dictionary form (*lemma*) using linguistic knowledge, taking into account the word's meaning and part of speech (e.g., *"better"* → *"good"*, *"running"* → *"run"*).







Natural Language Processing

What are embeddings ?

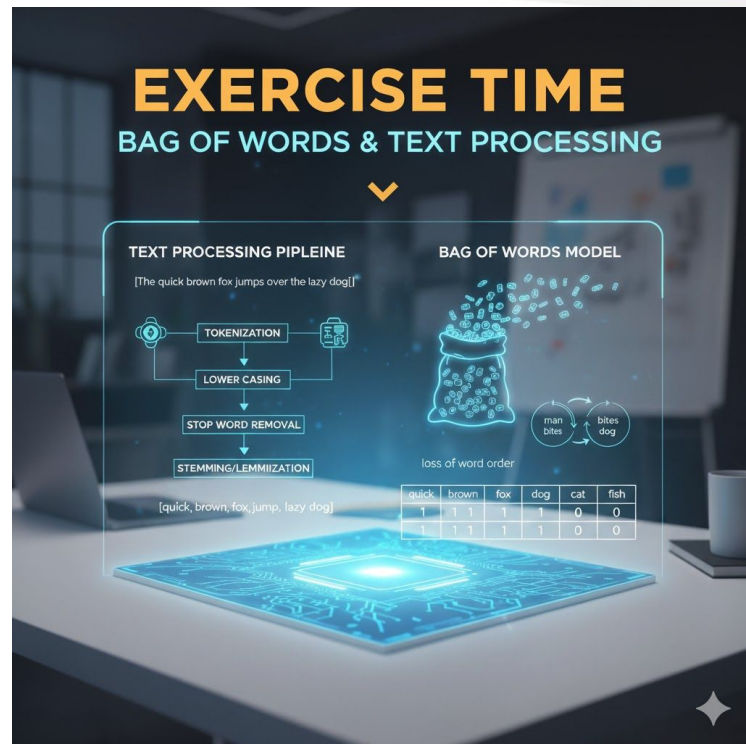
Embeddings are vectors that represent real-world objects, like **words**, images, or videos, in a form that **computer algorithms** like machine learning models can easily process.

	Feature #1	Feature #2	Feature #3	Feature #4	...
Document #1	1	0	1	0	0
Document #2	0	0	2	0	0
Document #3	0	1	0	0	1
Document #4	0	0	0	1	0
⋮	0	0	1	0	0

DTM

TL;DR it's just a bunch of numbers with meaning for computers

[Let, 's, do ,stuff]



Chapter 2.

Natural Language Processing

- | 1.1. What is NLP?
- | 1.2. Introduction to Search
- | 1.3. Processing text
- | 1.4. Representing text
- | 1.5. Similarity measures
- | 1.6. Next up



Are all documents, movies,... the same?

Is just the frequency of the word enough?

Are all documents, movies,... the same?

Is just the frequency of the word enough?

Doc 1: cats cats cats cats this is about dogs. Dogs like to bark.

Doc 2: Cats like to jump around

Are all documents, movies,... the same?

Is just the frequency of the word enough?

Doc 1: cats cats cats cats this is about dogs. Dogs like to bark.

Doc 2: Cats like to jump around

$V = \{\text{cats, this, is, about, dogs, like, to, bark, jump, around}\}$

Are all documents, movies,.. the same?

Is just the frequency of the word enough?

Doc 1: cats cats cats cats this is about dogs.
Dogs like to bark.

Doc 2: Cats like to jump around

$V = \{\text{cats, this, is, about, dogs, like, to, bark, jump, around}\}$



Are all documents, movies,.. the same?
No, the length of the document matters.
Is just the frequency of the word enough?
No. The loudest doesn't always win.



Are all documents, movies,.. the same?
No, the length of the document matters.
Is just the frequency of the word enough?
No. The loudest doesn't always win.

A possible solution?

TF-IDF = Term Frequency x Inverse
Document Frequency



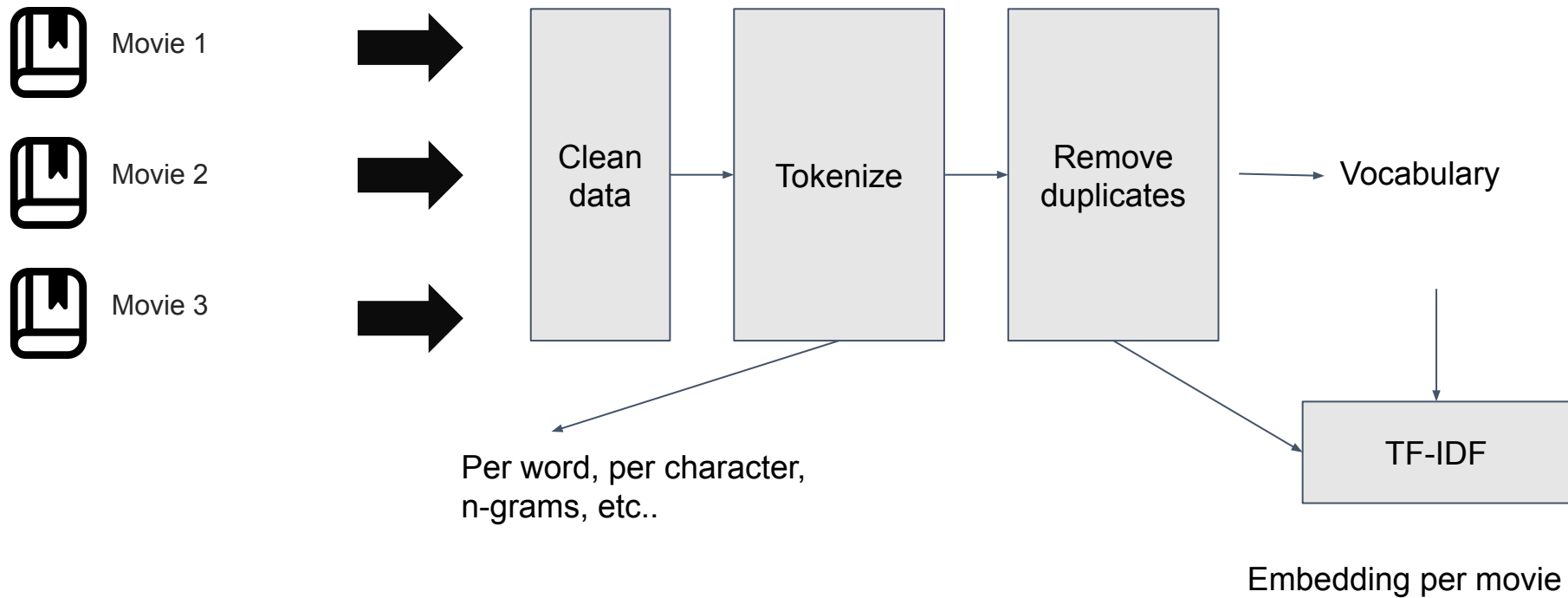
Are all documents, movies,.. the same?
No, the length of the document matters.
Is just the frequency of the word enough?
No. The loudest doesn't always win.

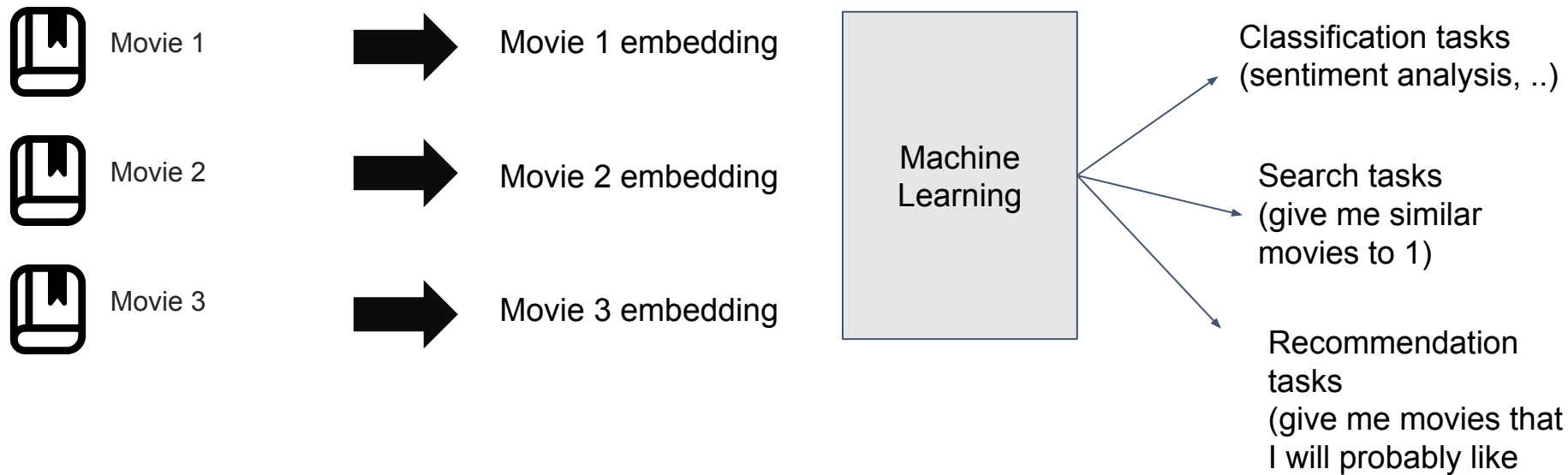
A possible solution?

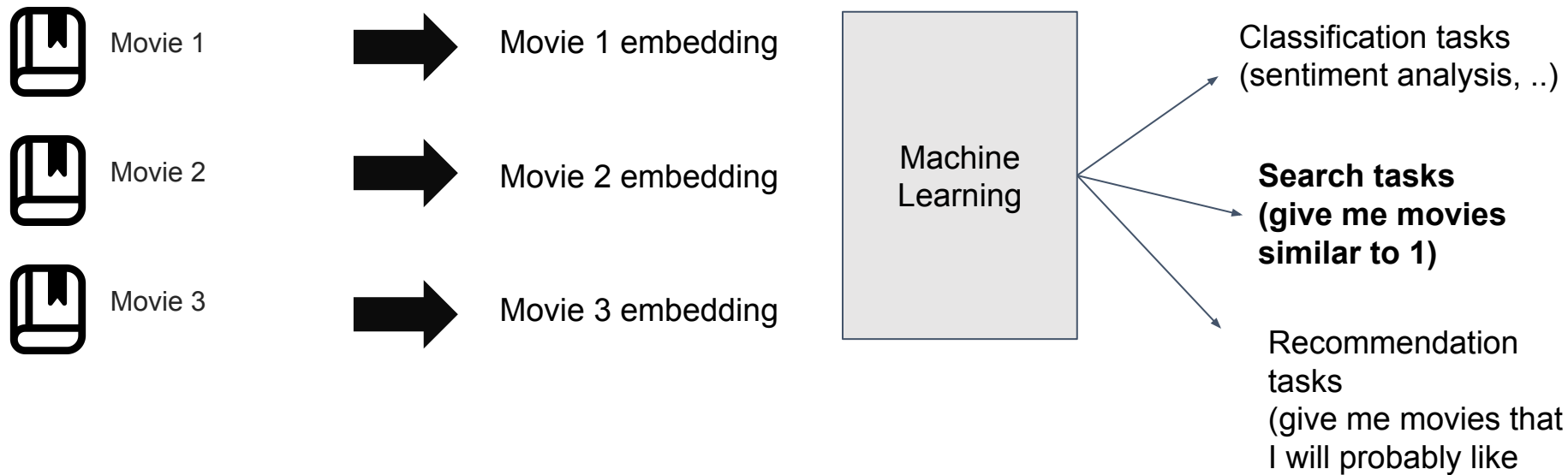
TF-IDF = Term Frequency x Inverse
Document Frequency

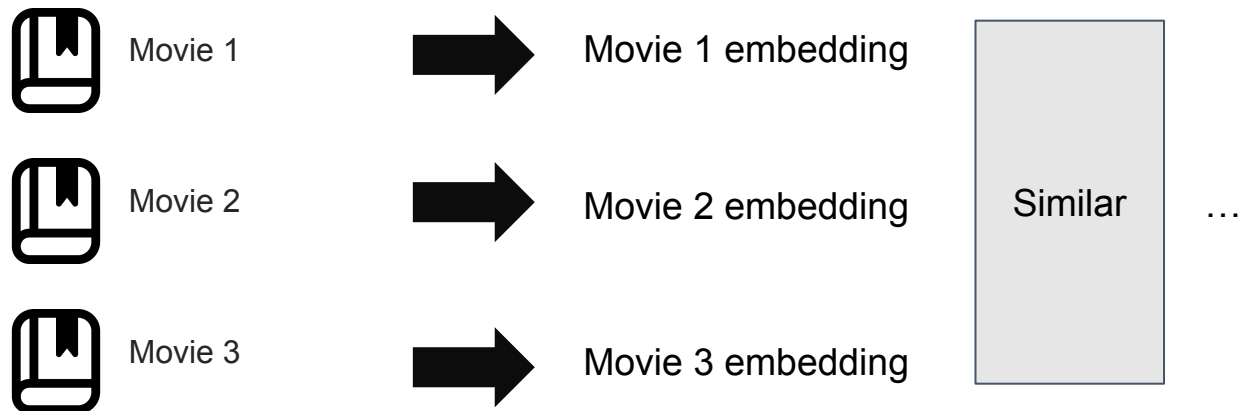
Term Frequency = Bag of Words (the
frequency of each word in the given
document)

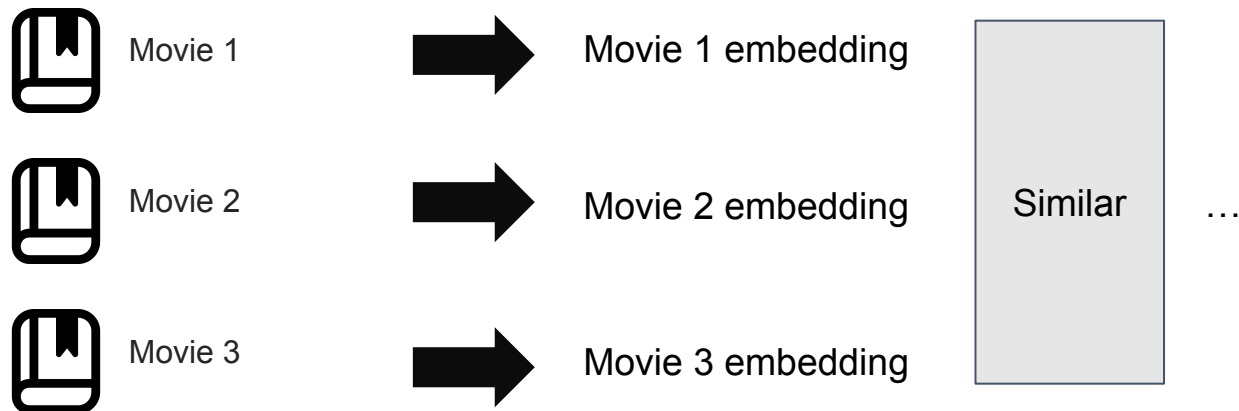
Inverse Document Frequency = How unique
a word is across **all** documents











Embedding = Vector

What are embeddings ?

Embeddings are vectors that represent real-world objects, like **words**, images, or videos, in a form that **computer algorithms** like machine learning models can easily process.

	Feature #1	Feature #2	Feature #3	Feature #4	...
Document #1	1	0	1	0	0
Document #2	0	0	2	0	0
Document #3	0	1	0	0	1
Document #4	0	0	0	1	0
⋮	0	0	1	0	0

DTM

TL;DR it's just a bunch of numbers with meaning for computers

Vector Space Model

Treat each document as a **point in space**, where mathematical tools can be used to measure distances and relationships.

Recall that the TF-IDF process converts a document into a list of weighted scores, one for every unique word in the vocabulary.

Definition: In a **Vector Space Model (VSM)**, a document is represented as a **vector**.

- Each **dimension** of the space corresponds to a unique word in the corpus vocabulary.
- The **value** along each dimension is the calculated TF-IDF weight for that word in the document.

{cats, this, is, around}. This means our VSM is a **10-dimensional space**.

Vector Space Model

Treat each document as a **point in space**, where mathematical tools can be used to **measure distances and relationships**.



Vector Space Model

Treat each document as a **point in space**, where mathematical tools can be used to **measure distances and relationships**.

Some similarities for exact nearest neighbor:

- Cosine Similarity
- Euclidean Similarity
- Jaccard Similarity

Vector Space Model

Treat each document as a **point in space**, where mathematical tools can be used to **measure distances and relationships**.

Some similarities for exact nearest neighbor:

- **Cosine Similarity**
- Euclidean Similarity
- Jaccard Similarity

$$\text{cosine similarity} = S_C(A, B) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}},$$

Cosine similarity measures **how similar two documents** are by comparing the angle between their vector representations in a **high-dimensional space**.

Each document is first converted into an **embedding**, a numerical vector capturing its *semantic meaning*.

The cosine similarity score, **which ranges from -1 to 1**, indicates **how closely the documents align in meaning**: a score near 1 means they are very similar, while a score near 0 means they are unrelated.

In document-to-document search, this **metric** allows systems to **find** and **rank** documents with content that is **semantically close**, regardless of differences in length or word choice, making it ideal for **clustering**, **recommendation**, and **information retrieval tasks**.

Note: Although cosine similarity theoretically ranges from -1 to $+1$, real-world document embeddings rarely produce negative values—most unrelated texts fall near 0.

<https://www.ibm.com/think/topics/cosine-similarity>

Vector Space Model

Treat each document as a **point in space**, where mathematical tools can be used to **measure distances and relationships**.

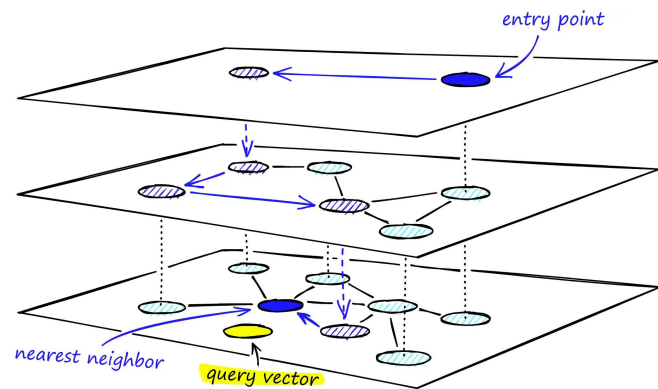
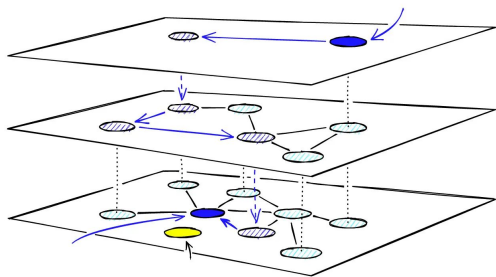
P.s in the real world with millions of documents, where speed and costs matter, we tend to use **Approximate Nearest Neighbors** (ANN) algorithms instead of **Exact Nearest Neighbors**, one of the most famous ones is the Hierarchical Navigable Small Worlds.

<https://www.pinecone.io/learn/a-developers-guide-to-ann-algorithms/>

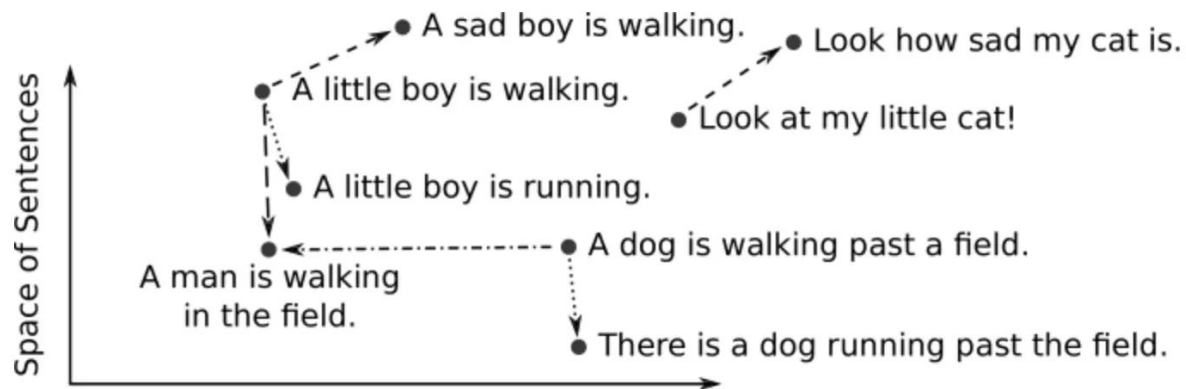
Hierarchical Navigable Small Worlds

Vector Search

Hierarchical Navigable Small Worlds (HNSW)



<https://www.pinecone.io/learn/series/faiss/hnsw/>



Formally speaking we could define search as:

$$F(q,c) = y$$

I.e a function that given a **query** q and a **corpus** c we get a set of **relevant** results y

Evaluation Metrics

- Precision (at K)
- Recall (at K)
- F1 Score
- NDCG

Evaluation Metrics

- Precision (at K)

Evaluation Metrics

- Precision (at K)

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

True Positives
i.e The system
classifies something
as true and they are
true

False Positives
i.e The system classifies something as
true and they are false

Evaluation Metrics

- Precision (at K)

Precision = $TP / (TP + FP)$

All items

True Positives
i.e The system
classifies them as true
and they are true

False Positives
i.e The system classifies them as true
and they are false

Evaluation Metrics

- Precision (at K)


$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

% of times the system got the true label right

Evaluation Metrics

- Precision (at K) is the proportion of recommended items in the top-k set that are relevant



$$\text{Precision at K} = \text{TP at K} / (\text{TP at K} + \text{FP at K})$$

Evaluation Metrics

- Precision (at K) is the proportion of recommended items in the top-k set that are relevant



$$\text{Precision at K} = \text{TP at K} / (\text{TP at K} + \text{FP at K})$$

Result A	Relevant
Result B	Relevant
Result C	Not Relevant

Evaluation Metrics

- Precision (at K) is the proportion of recommended items in the top-k set that are relevant



Precision at K = $TP \text{ at } K / (TP \text{ at } K + FP \text{ at } K)$ Precision at 3 = $\frac{2}{3} = 66\%$

Result A	Relevant
Result B	Relevant
Result C	Not Relevant

Evaluation Metrics

- Precision (at K) is the proportion of recommended items in the top-k set that are relevant

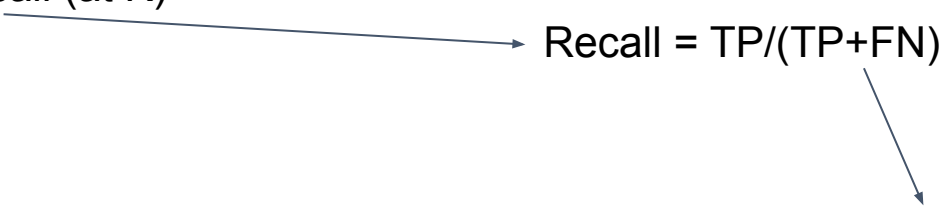


Precision at K = $TP \text{ at } K / (TP \text{ at } K + FP \text{ at } K)$ Precision at 3 = $\frac{2}{3} = 66\%$

Result A	Relevant
Result B	Not Relevant
Result C	Relevant

Evaluation Metrics

- Recall (at K)


$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

False Negatives
i.e The system classifies
something as false and they
are true

Evaluation Metrics

- Recall (at K) (also known as sensitivity) is the fraction of relevant instances that were retrieved.



Recall at K = $TP \text{ at } K / (TP \text{ at } K + FN \text{ at } K)$
Relevant items = 5

Recall at 3 = $2/2 = 100\%$

Result A	Relevant
Result B	Not Relevant
Result C	Relevant

Evaluation Metrics

- Recall (at K) (also known as sensitivity) is the fraction of relevant instances that were retrieved.



Recall at K = $TP \text{ at } K / (\text{All relevant items})$
Relevant items = 5

Recall at 3 = $\frac{2}{5} = 40\%$

Result A	Relevant
Result B	Not Relevant
Result C	Relevant

Evaluation Metrics

- Recall (at K) (also known as sensitivity) is the **fraction of relevant instances that were retrieved.**



Recall at K = TP at K / (All relevant items)
Relevant items = 5

Recall at 3 = $\frac{2}{5} = 40\%$

Result A	Relevant
Result B	Not Relevant
Result C	Relevant

Natural Language Processing

Search

Evaluation Metrics

- F1 Score is a metric that takes into account both precision and recall to provide a balanced evaluation of a system's performance

Evaluation Metrics

- F1 Score is a metric that takes into account both precision and recall to provide a balanced evaluation of a system's performance

$$\text{F1 Score} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

Evaluation Metrics

- F1 Score is a metric that takes into account both precision and recall to provide a balanced evaluation of a system's performance

$$\text{F1 Score} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

... gives more weight to the lower of the two values. This means that if **either precision or recall is low** (i.e., the weaker of the two metrics), **the harmonic mean will also be low**, *reflecting the fact that the system is not performing well in at least one of these aspects*. It penalizes systems that have an extreme imbalance between precision and recall.

Evaluation Metrics

- F1 Score at K is a metric that takes into account both precision and recall to provide a balanced evaluation of a system's performance

$$\text{F1 Score at K} = 2 * (\text{Precision at K} * \text{Recall at K}) / (\text{Precision at K} + \text{Recall at K})$$

... gives more weight to the lower of the two values. This means that if **either precision or recall is low** (i.e., the weaker of the two metrics), **the harmonic mean will also be low**, *reflecting the fact that the system is not performing well in at least one of these aspects*. It penalizes systems that have an extreme imbalance between precision and recall.

Natural Language Processing

Search

Evaluation Metrics

- Normalized Discounted Cumulative Gain (NDCG)

Evaluation Metrics

- Normalized Discounted Cumulative Gain (NDCG) assesses how **well the top-ranked items in a list align with the preferences or relevance judgments of users**, i.e order matters.

Evaluation Metrics

- Normalized **Discounted Cumulative Gain** (NDCG).

$$\text{DCG}_p = \sum_{i=1}^p \frac{rel_i}{\log_2(i+1)} =$$

Evaluation Metrics

- Normalized **Discounted Cumulative Gain (NDCG)**.

$$\text{DCG}_p = \sum_{i=1}^p \frac{rel_i}{\log_2(i+1)}$$

DCG_p is the DCG at position p.

Evaluation Metrics

- Normalized **Discounted Cumulative Gain (NDCG)**.

$$\text{DCG}_p = \sum_{i=1}^p \frac{rel_i}{\log_2(i+1)}$$

- DCG_p is the DCG at position p .
- rel_i is the relevance score of the item at position i in the ranking list (typically a non-negative number, where higher values represent higher relevance).

Evaluation Metrics

- Normalized **Discounted Cumulative Gain (NDCG)**.

$$\text{DCG}_p = \sum_{i=1}^p \frac{rel_i}{\log_2(i+1)}$$

- DCG_p is the DCG at position p .
- rel_i is the relevance score of the item at position i in the ranking list (typically a non-negative number, where higher values represent higher relevance).
- Log is used to produce a smooth reduction

Evaluation Metrics

- Normalized **Discounted Cumulative Gain (NDCG)**.

The premise of DCG is that **highly relevant documents appearing lower in a search result list should be penalized** as the graded relevance value is reduced logarithmically proportional to the position of the result.

Natural Language Processing

Search

Evaluation Metrics

- **Normalized Discounted Cumulative Gain (NDCG).**

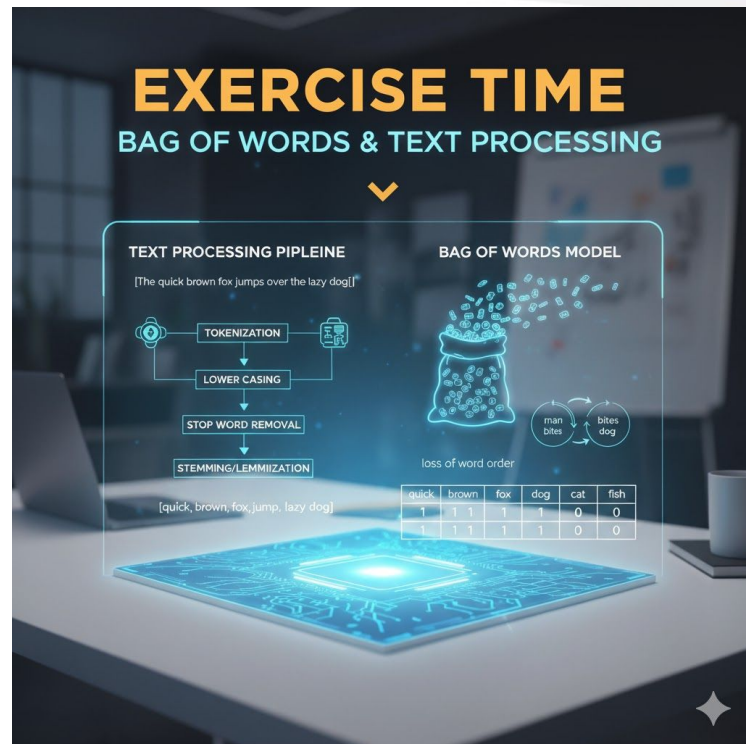
$$\text{nDCG}_p = \frac{DCG_p}{IDCG_p}$$

Diagram illustrating the components of NDCG:

- DCG_p (Discounted Cumulative Gain at position p) is the numerator.
- $IDCG_p$ (Ideal Discounted Cumulative Gain at position p) is the denominator, representing the ideal performance.
- The formula for $IDCG_p$ is:
$$IDCG_p = \sum_{i=1}^{|REL_p|} \frac{rel_i}{\log_2(i+1)}$$
 - $|REL_p|$ represents the list of relevant documents (ordered by their relevance) in the corpus up to position p .
 - rel_i is the relevance of the document at position i .

ideal discounted cumulative gain

[Let, 's, do ,stuff, again]



[Let, 's, do ,stuff, again]

- **Implement TF-IDF from Scratch** - Build TF-IDF calculation manually to understand how term weighting works ($TF \times IDF$)
- **Implement Similarity Search** - Use TF-IDF vectors with cosine similarity to rank and retrieve relevant documents
- **Explore Hybrid Search** - Combine TF-IDF similarity search with keyword matching for better search results
- **Use Industry Tools** - Learn NLTK, spaCy, and scikit-learn for production-ready preprocessing and search systems
- *RAG (Bonus)* - Retrieval-Augmented Generation combining search with LLM generation for Q&A systems